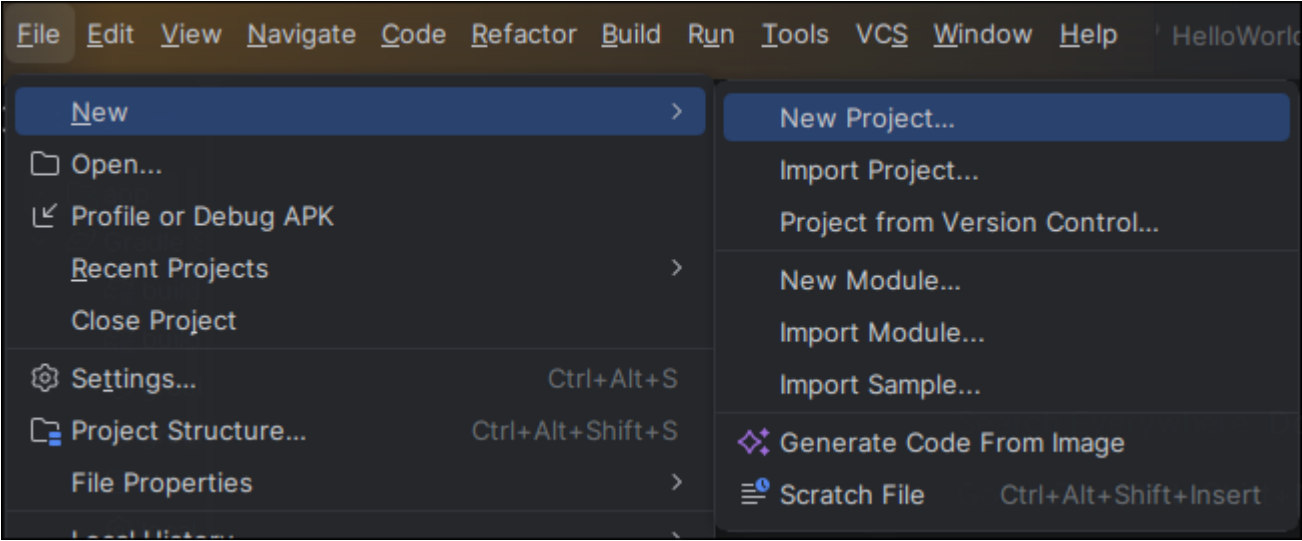


	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

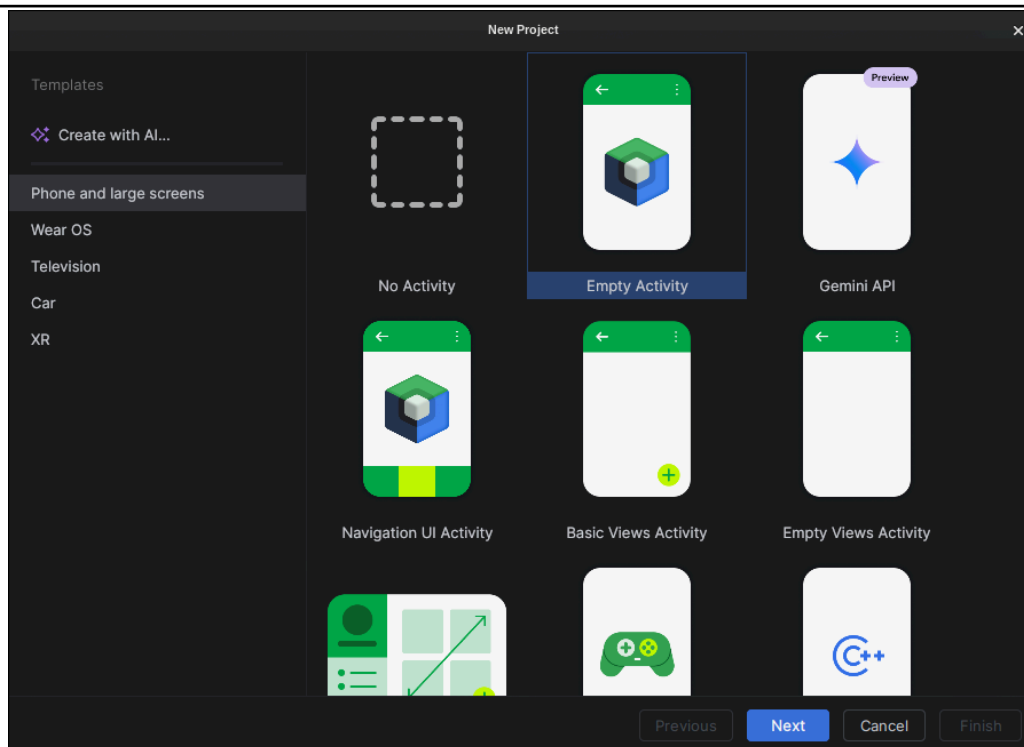
INFORME DE LABORATORIO

(formato estudiante)

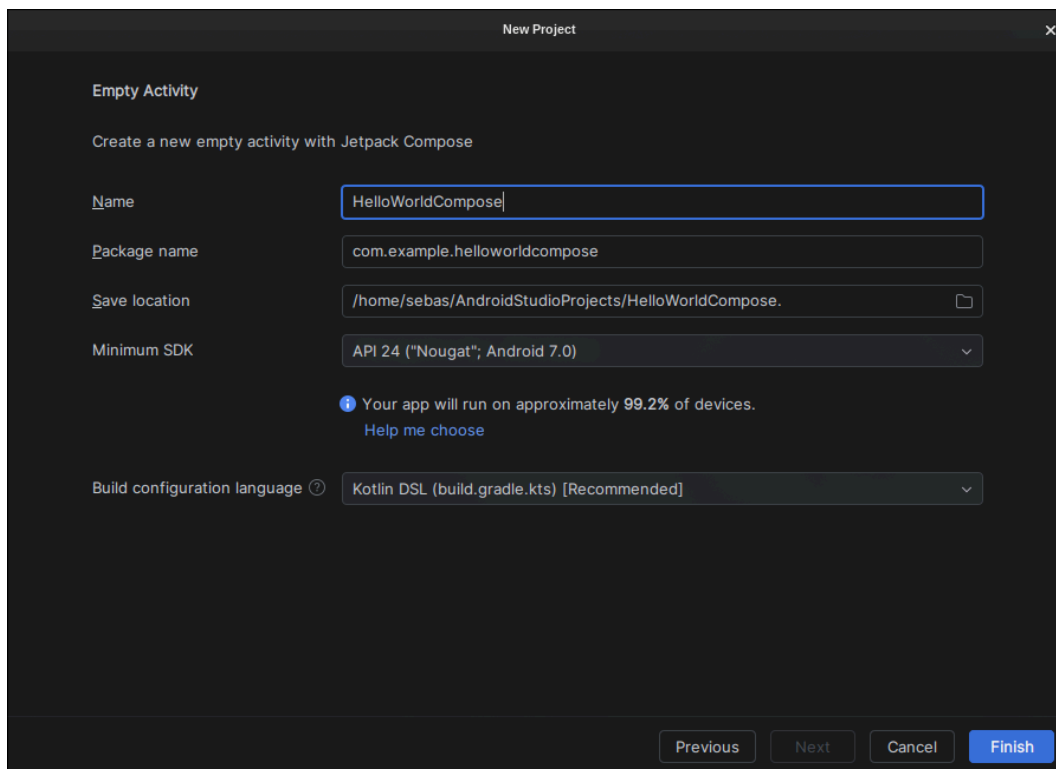
INFORMACIÓN BÁSICA					
ASIGNATURA:	INTRODUCCIÓN AL DESARROLLO DE NUEVAS PLATAFORMAS (E)				
TÍTULO DE LA PRÁCTICA:	Primeros pasos con Android Studio y Jetpack Compose				
NÚMERO DE PRÁCTICA:	2	AÑO LECTIVO:	2026-B	NRO. SEMESTRE:	VIII
FECHA DE PRESENTACIÓN	12/09/2026	HORA DE PRESENTACIÓN	23:59		
INTEGRANTE (s): - Zuñiga Villacorta, Peter Sebastian				NOTA (0-20)	
DOCENTE(s): Roxana Evelyn Limache Calatayud					

RESULTADOS Y PRUEBAS
Github Repo: https://github.com/UNSA-IS-2026-B/idnp_lab/tree/main/lab02 Replicar el ejercicio resuelto por el docente (proyecto Compose + Hello World). Paso 1. Abrir Android Studio. En la pantalla de bienvenida, seleccionar "New Project".  Paso 2. En la lista de plantillas, elegir "Empty Activity" (dentro de la categoría Phone and Tablet). Esta plantilla ya viene configurada con Jetpack Compose.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 2

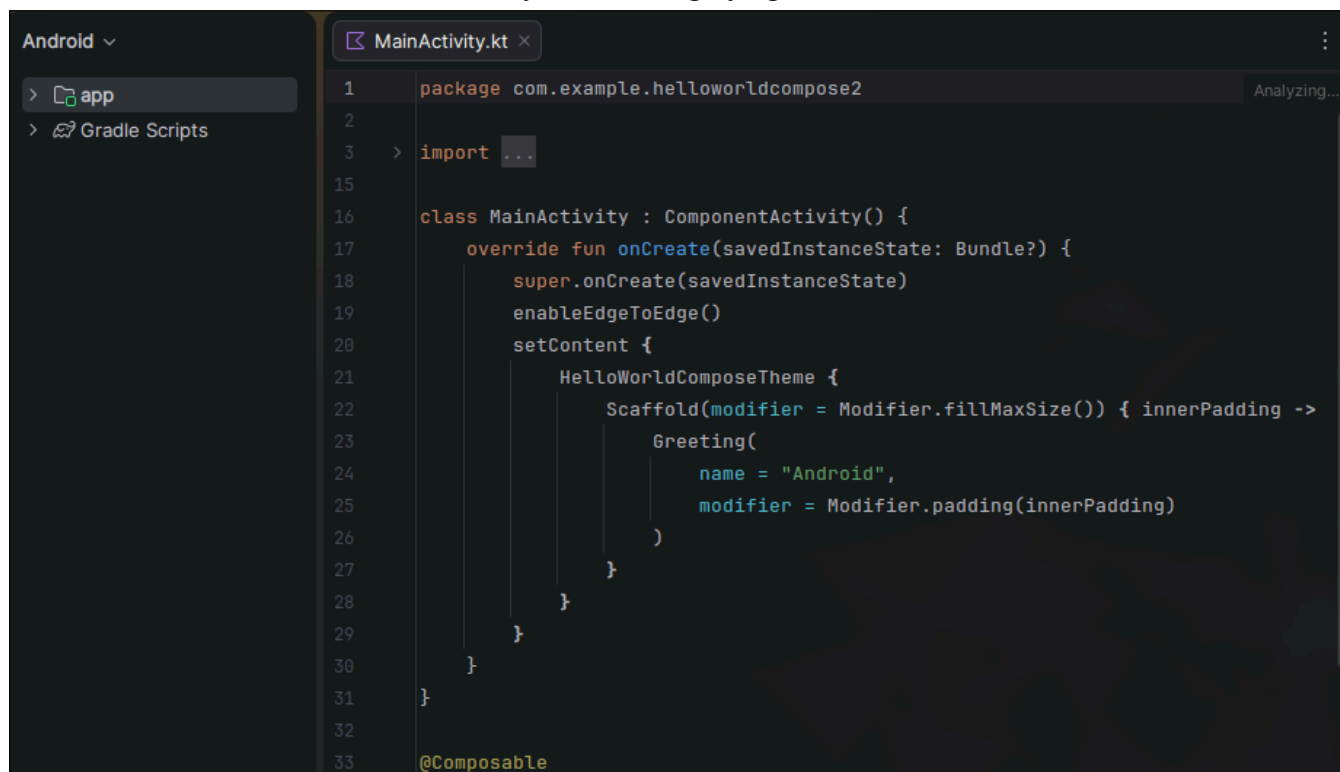


Paso 3. Configurar el nombre del proyecto ("HelloWorldCompose"), el nombre del paquete y confirmar que el lenguaje sea Kotlin. Presionar "Finish".



	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 3

Paso 4. Esperar a que Android Studio descargue dependencias y sincronice Gradle. Se abrirá automáticamente el archivo MainActivity.kt con código ya generado.

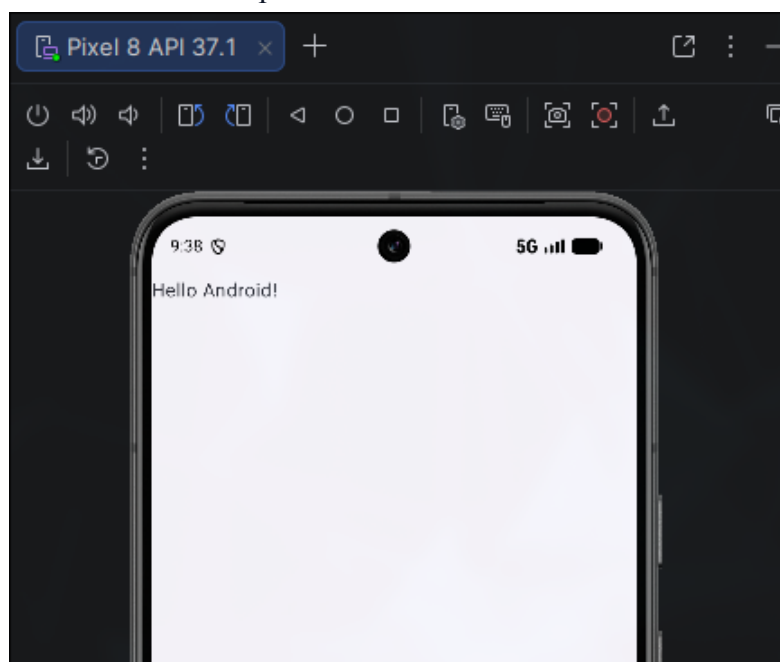


```

1 package com.example.helloworldcompose2
2
3 import ...
4
15
16 class MainActivity : ComponentActivity() {
17     override fun onCreate(savedInstanceState: Bundle?) {
18         super.onCreate(savedInstanceState)
19         enableEdgeToEdge()
20         setContent {
21             HelloWorldComposeTheme {
22                 Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
23                     Greeting(
24                         name = "Android",
25                         modifier = Modifier.padding(innerPadding)
26                     )
27                 }
28             }
29         }
30     }
31 }
32
33 @Composable

```

Paso 5. Ejecutar el proyecto presionando el botón Run (con un emulador Pixel 8 conectado). Debe aparecer el mensaje "Hello Android!" en pantalla.



	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 4

Paso 6. Dentro de MainActivity.kt, ubicar la función `@Composable` llamada `Greeting(name: String, ...)`. Esta función recibe un texto y lo muestra mediante un componente `Text`.

```

33     @Composable
34     fun Greeting(name: String, modifier: Modifier = Modifier) {
35         Text(
36             text = "Hello $name!",
37             modifier = modifier
38         )
39     }

```

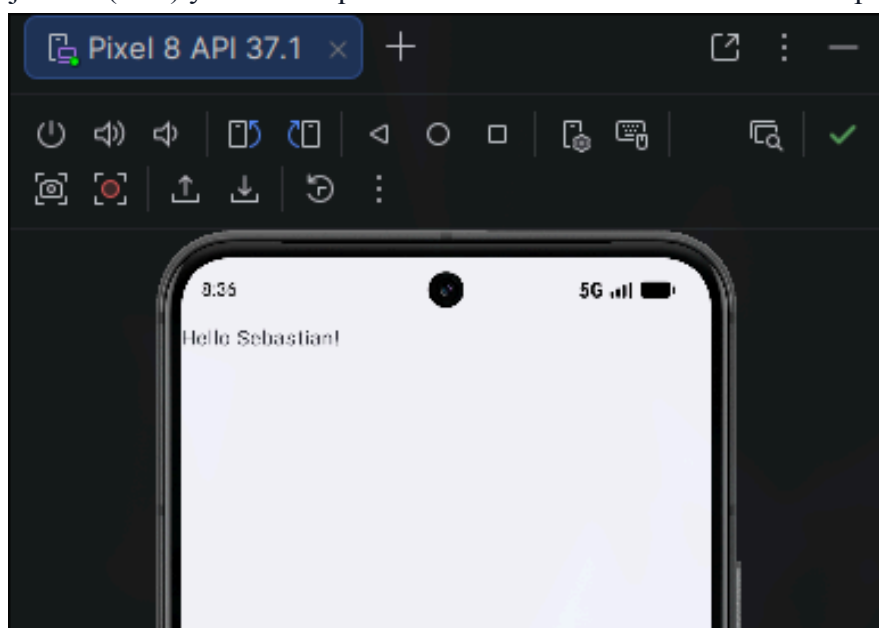
Paso 7. Ubicar la llamada `Greeting(name = "Android", ...)` dentro de `setContent { }`, y reemplazar el texto "Android" por el del estudiante.

```

setContent {
    HelloWorldComposeTheme {
        Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
            Greeting(
                name = "Sebastian",
                modifier = Modifier.padding(innerPadding)
            )
        }
    }
}

```

Paso 8. Volver a ejecutar (Run) y verificar que el emulador ahora muestra el saludo personalizado.



	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 5

Luego, implementar una aplicación sencilla para registrar el libro que el estudiante está leyendo actualmente. La aplicación debe permitir lo siguiente:

- 1. Una pantalla (Composable) con tres campos de texto (TextField): título del libro, autor y número de páginas leídas, y dos botones: "Guardar" y "Ver registro".**

Se construyó la interfaz requerida y la lógica de almacenamiento interno mediante la función @Composable RegistroLibroScreen(). Se utilizaron variables de estado con remember y mutableStateOf para vincular dinámicamente los componentes OutlinedTextField y actualizar la pantalla en tiempo real:

Kotlin

```
// Estados para almacenar los campos de texto
var titulo by remember { mutableStateOf("") }
var autor by remember { mutableStateOf("") }
var paginas by remember { mutableStateOf("") }

// Estado para mostrar la lectura en pantalla
var contenidoGuardado by remember { mutableStateOf("") }

Column(
    modifier = Modifier
        .fillMaxSize()
        .padding(16.dp),
    horizontalAlignment = Alignment.CenterHorizontally,
    verticalArrangement = Arrangement.spacedBy(12.dp)
) {
    Text(
        text = "Registro de Libro",
        style = MaterialTheme.typography.headlineMedium
    )

    OutlinedTextField(
        value = titulo,
        onChange = { titulo = it },
        label = { Text("Título del libro") },
        modifier = Modifier.fillMaxWidth()
    )

    OutlinedTextField(
        value = autor,
        onChange = { autor = it },
        label = { Text("Autor") },
        modifier = Modifier.fillMaxWidth()
    )

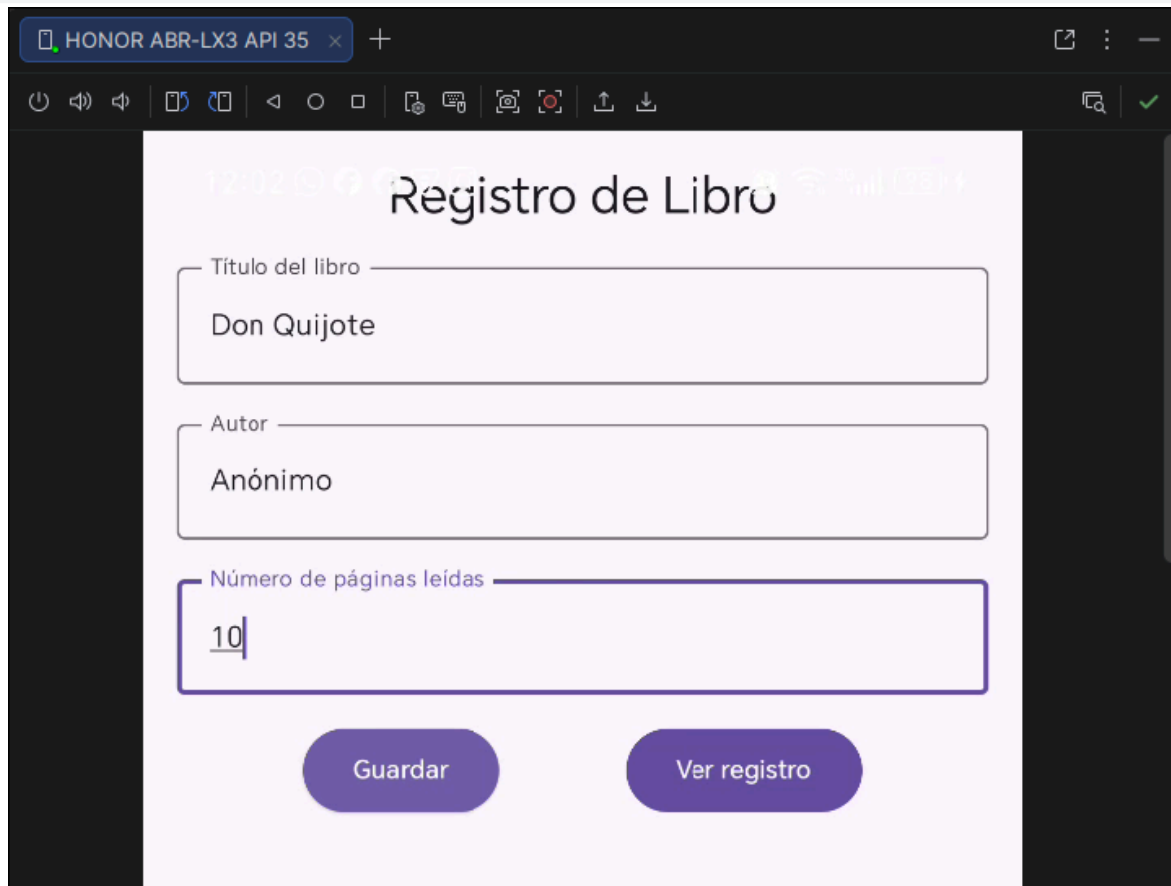
    OutlinedTextField(
        value = paginas,
        onChange = { paginas = it },
        label = { Text("Número de páginas leídas") },
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 6

```

        modifier = Modifier.fillMaxWidth()
    )

```



2. Al presionar "Guardar", escribir los tres datos ingresados en un archivo de texto plano guardado en el almacenamiento interno de la aplicación (openFileOutput, modo **MODE_PRIVATE**).

Al presionar Guardar, se concatenan las variables y se escriben en el archivo plano libro_registro.txt utilizando context.openFileOutput() bajo el modo privado Context.MODE_PRIVATE:

```

Kotlin
Button(onClick = {
    val textoAGuardar = "Título: $titulo\nAutor: $autor\nPáginas: $paginas"
    try {
        // Guarda el texto en el almacenamiento interno de la app
        context.openFileOutput("libro_registro.txt", Context.MODE_PRIVATE).use { output
->
            output.write(textoAGuardar.toByteArray())
        }
    }
}

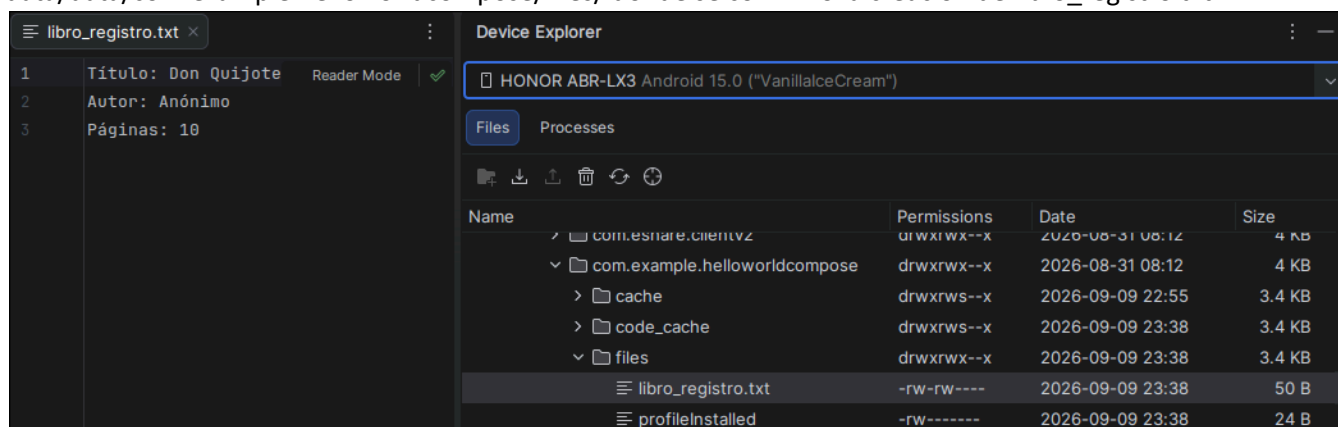
```

```

// Limpieza de campos tras guardar
titulo = ""; autor = ""; paginas = ""
} catch (e: Exception) {
    Log.e("RegistroLibro", "Error al guardar el archivo", e)
}
}) {
    Text("Guardar")
}

```

Mediante la ventana Device File Explorer se navegó hasta la ruta data/data/com.example.helloworldcompose/files/ donde se confirmó la creación de libro_registro.txt.



3. Al presionar "Ver registro", leer el archivo guardado y mostrar su contenido por consola empleando la clase Log (Log.d).

Al presionar Ver registro, se lee el archivo con `openFileInput()`, se envía el contenido a la consola con `Log.d()` y se asigna el texto a la variable `contenidoGuardado` para actualizar el componente `Text`:

```

Kotlin
Button(onClick = {
    try {
        // Lectura del archivo plano
        val textoLeido = context.openFileInput("libro_registro.txt").bufferedReader().use {
            it.readText()
        }

        // Imprimir en Logcat
        Log.d("RegistroLibro", "Contenido del archivo:\n$textoLeido")

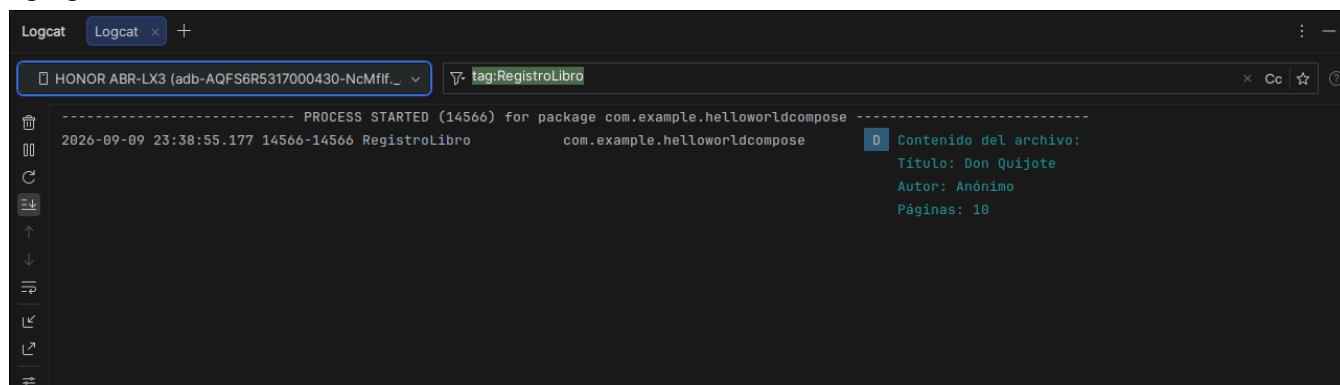
        // Mostrar en la pantalla actualizando el estado
        contenidoGuardado = textoLeido
    } catch (e: Exception) {
        Log.e("RegistroLibro", "Error al leer el archivo", e)
        contenidoGuardado = "No hay datos guardados aún."
    }
}) {
    Text("Ver registro")
}

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 8

}

Con la etiqueta tag:RegistroLibro configurada en Logcat, se obtuvo la impresión del archivo en la laptop:



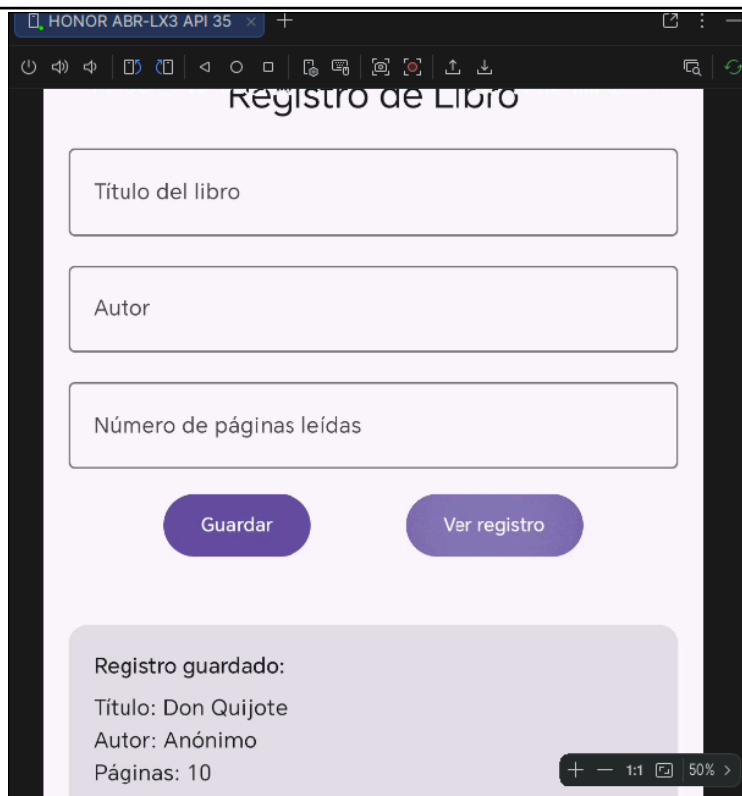
4. Además de mostrarlo por consola, mostrar el registro leído en pantalla empleando un Text.

Kotlin

```
// Muestra el registro leído en pantalla
if (contenidoGuardado.isNotEmpty()) {
    Card(
        modifier = Modifier.fillMaxWidth()
    ) {
        Column(modifier = Modifier.padding(16.dp)) {
            Text(
                text = "Registro guardado:",
                style = MaterialTheme.typography.titleMedium
            )
            Spacer(modifier = Modifier.height(6.dp))
            Text(text = contenidoGuardado)
        }
    }
}
```

Al presionar Guardar con los campos llenos y posteriormente presionar Ver registro, la tarjeta inferior despliega los datos leídos.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 9



CUESTIONARIO:

1. **¿Consideras complejo el entorno de Android Studio y el uso de Jetpack Compose para tu primera aplicación? Explica brevemente por qué.**

No resulta un entorno excesivamente complejo gracias al paradigma declarativo de Jetpack Compose. A diferencia de lo tradicional que requiere estructurar vistas en archivos XML independientes, Compose permite construir la interfaz de usuario mediante funciones `@Composable` escritas directamente en Kotlin, lo que reduce el código repetitivo y agiliza el desarrollo [1], [3]. La principal curva de aprendizaje inicial implica comprender la gestión del estado reactivo y la configuración de dependencias en Gradle [2].

2. **¿Cuál es el propósito de la función `setContent {}` dentro del `onCreate()` de una Activity que usa Jetpack Compose?**

Su propósito es establecer el bloque de contenido principal de la Activity, permitiendo invocar las funciones `@Composable` que definirán la interfaz gráfica [3]. Reemplaza al método clásico `setContentView(R.layout...)` basado en layouts XML, vinculando la ventana de la aplicación con la jerarquía de vistas declarativas de Jetpack Compose [1], [2].

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 10

CONCLUSIONES

1. Se identificó la arquitectura base de un proyecto en Android Studio, comprendiendo a MainActivity.kt como la clase principal de entrada y los archivos de Gradle como los gestores de dependencias y configuración.
2. Se construyó una interfaz de usuario limpia mediante componentes @Composable (Column, Row, OutlinedTextField, Button y Text), permitiendo estructurar el formulario de lectura de forma rápida y modular.
3. Se aplicó el uso de remember y mutableStateOf para capturar en tiempo real las entradas de texto del usuario y actualizar dinámicamente la pantalla durante las operaciones de guardado y lectura.

REFERENCIAS Y BIBLIOGRAFÍA

- [1] Android Developers, "Introducción al desarrollo de aplicaciones Android". [En línea]. Disponible: <https://developer.android.com/>
- [2] Material y apuntes de Jetpack Compose | Android Studio – GitHub. [En línea]. Disponible: <https://github.com/empm/JetpackCompose>
- [3] Android Developers, "Create your first Android app". [En línea]. Disponible: <https://developer.android.com/training/basics/firstapp>
- [4] Material Design, "Introduction". [En línea]. Disponible: <https://m3.material.io/>
- [5] SwiftUI Learning – GitHub. [En línea]. Disponible: <https://github.com/slekens/SwiftUI-Learning>
- [6] “Tipos de apps. Diferencias entre nativas, híbridas y web”, video. [En línea]. Disponible: <https://www.youtube.com/watch?v=9cOzoGkUnVM>